

1. Setup e Imports

```
import json as _json
import json
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm import tqdm
import os
import sys
from pathlib import Path
import subprocess

# Agregar path del proyecto para imports
sys.path.append(os.path.abspath('../..'))
from sae.tools.naming_utils import save_checkpoint, get_model_name, get_extras_id, get_checkpoint_dir, get_model_dir
from sae.tools.experiment_utils import get_experiment_dir, get_plots_dir, get_metrics_dir, get_logs_dir

# Configuración de visualización para Quarto/PDF
pd.set_option('display.width', 100)
pd.set_option('display.max_columns', None)
np.set_printoptions(linewidth=100, precision=4, suppress=True)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Dispositivo: {device}')
```

Dispositivo: cuda

2 Metadata para Naming de Checkpoints

```
# Metadata para naming de checkpoints (separada de hiperparámetros)
NAMING_META = {
    'variante': 'matryoshkabatchtopk',
    'tecnica': 'matryoshkabatchtopk',
    'capa': 6,
    'juegos': 25000,
    'base_path': 'C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt',
    'experiment_base_path': os.path.abspath('../../../experiments'),
    'extras': {
        'expansion': 16,
        'k': 81,
        'matryoshka_widths': [2048, 4096, 8192],
        'epochs': 1000,
        'patience': 20,
        'lr': 1e-3
    }
}

print('\nMetadata de naming:')
for key, value in NAMING_META.items():
    print(f' {key}: {value}')
```

Metadata de naming:

```
variante: matryoshkabatchtopk
tecnica: matryoshkabatchtopk
capa: 6
juegos: 25000
base_path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt
experiment_base_path: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments
extras: {'expansion': 16, 'k': 81, 'matryoshka_widths': [2048, 4096, 8192], 'epochs': 1000
```

```
extras_id = get_extras_id(
    get_checkpoint_dir(
        NAMING_META['base_path'],
        NAMING_META['variante'],
        NAMING_META['tecnica'],
        NAMING_META['capa'],
        NAMING_META['juegos']
```



```

NUM_EPOCHS = NAMING_META['extras']['epochs']

cfg = {
    'architecture': 'matryoshkabatchtopk',
    'd_in': 512,
    'd_sae': 8192,
    'k': 81,
    'matryoshka_widths': [2048, 4096, 8192],
    'lr': 1e-3,
    'train_batch_size_tokens': 4096,
    'hook_name': 'blocks.5.hook_resid_post',
    # P3: betas según config.py de los autores (beta2=0.99 vs default PyTorch 0.999)
    'beta1': 0.9,
    'beta2': 0.99,
    # P1: parámetros para auxiliary loss de dead features (config.py de los autores)
    'n_batches_to_dead': 20, # batches sin activación para declarar feature muerta
    'top_k_aux': 512, # top-k features muertas usadas en L_aux
    'aux_penalty': 1/32, # peso de L_aux (paper ec. 5)
    # P2: normalización del input por muestra
    'input_unit_norm': True,
}

cfg_summary = pd.DataFrame({
    'Parámetro': ['architecture', 'd_in', 'd_sae', 'k (L0 promedio)', 'matryoshka_widths',
                  'hook_name', 'lr', 'beta1', 'beta2',
                  'n_batches_to_dead', 'top_k_aux', 'aux_penalty', 'input_unit_norm'],
    'Valor': [cfg['architecture'], cfg['d_in'], cfg['d_sae'], cfg['k'],
              str(cfg['matryoshka_widths']), cfg['hook_name'], cfg['lr'],
              cfg['beta1'], cfg['beta2'],
              cfg['n_batches_to_dead'], cfg['top_k_aux'], cfg['aux_penalty'],
              cfg['input_unit_norm']]
})

print('Configuración SAELens:')
print(cfg_summary.to_string(index=False))

```

Configuración SAELens:

Parámetro	Valor
architecture	matryoshkabatchtopk
d_in	512
d_sae	8192
k (L0 promedio)	81
matryoshka_widths	[2048, 4096, 8192]

hook_name	blocks.5.hook_resid_post
lr	0.001
beta1	0.9
beta2	0.99
n_batches_to_dead	20
top_k_aux	512
aux_penalty	0.03125
input_unit_norm	True

4. Dataset de Activaciones (Train/Validation Split 80-20%)

Activaciones pre-extraídas con `sae/activations/run_extraction.py` desde la capa `blocks.5.hook_resid_post`.

```
class ActivationsDataset(Dataset):
    """Dataset para cargar activaciones extraídas"""
    def __init__(self, activations_path):
        self.activations = np.load(activations_path, mmap_mode='r')
    def __len__(self):
        return len(self.activations)
    def __getitem__(self, idx):
        return torch.tensor(self.activations[idx], dtype=torch.float32)

activations_file = Path(f"../../activations/data/layer{NAMING_META['capa']}_{NAMING_META['hook']}")
full_dataset = ActivationsDataset(activations_file)

# Split 80-20 reproducible
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(
    full_dataset, [train_size, val_size],
    generator=torch.Generator().manual_seed(42)
)

train_loader = DataLoader(train_dataset, batch_size=cfg["train_batch_size_tokens"], shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=cfg["train_batch_size_tokens"], shuffle=False)

dataset_info = pd.DataFrame({
    'Conjunto': ['Train', 'Validación', 'Total'],
    'Muestras': [f'{len(train_dataset):,}', f'{len(val_dataset):,}', f'{len(full_dataset):,}'],
    'Shape': [str(full_dataset.activations.shape), '', ''],
})
```

```
})
print(dataset_info.to_string(index=False))
```

```

Conjunto  Muestras      Shape
Train 1,180,000 (1475000, 512)
Validación  295,000
Total 1,475,000

```

5. Modelo: MatryoshkaBatchTopK SAE

Implementación de la arquitectura MatryoshkaBatchTopK según el paper (ec. 3-5) y el código de los autores (`sae.py`).

Arquitectura: - El encoder proyecta **una sola vez** con W_{enc} completo ($d_{in} \rightarrow d_{sae}$) para obtener $f(x)$ de tamaño d_{sae} (paper ec. 3) - **BatchTopK global único:** se seleccionan los $k \times batch_size$ valores más altos sobre los $d_{sae} \times batch_size$ valores del batch completo — un único threshold compartido entre todos los anchos (paper ec. 6) - Las reconstrucciones por nivel usan **prefijos del mismo vector:** `all_acts_topk[:, 0:w]` — las features 0:2048 son literalmente las mismas en el nivel $w=2048$ y en el nivel $w=4096$ (paper ec. 4) - En **eval**, se usa un threshold persistente calibrado con EMA durante training — no se recalcula (paper ec. 7) - **L0 es variable por muestra pero promedia k en cada batch**

Parámetro	L1 (standard)	BatchTopK	Matryoshka-BatchTopK
Sparsity	<code>sparsity_coef</code> (soft)	k (promedio del batch)	k (promedio, threshold global)
L0	Variable	Variable por muestra	Variable por muestra
Loss	$MSE + \cdot L1$	MSE	$\Sigma MSE_width + \cdot L_aux$

```

class MatryoshkaBatchTopKSAE(nn.Module):
    """
    Sparse Autoencoder con activación MatryoshkaBatchTopK.
    Corregido según paper (ec. 3-5) e implementación de los autores (sae.py).

    Correcciones aplicadas:
    - P1: encoding único con  $W_{enc}$  completo; BatchTopK global; prefijos compartidos
    - P1: BatchTopK preserva valores originales ( $x * indicator$ ), threshold EMA para eval

```

```

- P1: auxiliary loss para dead features (sae.py:198-221)
- P2: make_decoder_weights_and_grad_unit_norm proyecta gradiente (sae.py:51-57)
- P2: preprocessing input_unit_norm configurable (sae.py:36-43)
- P3: W_dec inicializado como transpuesta de W_enc (sae.py:93-94)
"""
def __init__(self, d_in, d_sae, k, matryoshka_widths,
              n_batches_to_dead=20, top_k_aux=512, aux_penalty=1/32,
              input_unit_norm=False):
    super().__init__()
    self.d_in = d_in
    self.d_sae = d_sae
    self.k = k
    self.matryoshka_widths = sorted(matryoshka_widths)
    assert self.matryoshka_widths[-1] == d_sae, \
        f"El último ancho ({self.matryoshka_widths[-1]}) debe ser d_sae ({d_sae})"

    self.n_batches_to_dead = n_batches_to_dead
    self.top_k_aux = top_k_aux
    self.aux_penalty = aux_penalty
    self.input_unit_norm = input_unit_norm

    self.W_enc = nn.Parameter(torch.empty(d_in, d_sae))
    self.W_dec = nn.Parameter(torch.empty(d_sae, d_in))
    self.b_dec = nn.Parameter(torch.zeros(d_in))

    # P3: W_dec = W_enc^T normalizado (sae.py:93-94) - inicialización tied
    nn.init.kaiming_uniform_(self.W_enc)
    self.W_dec.data[:] = self.W_enc.t().data
    self.W_dec.data[:] = self.W_dec / self.W_dec.norm(dim=-1, keepdim=True)

    # P1: threshold persistente calibrado con EMA durante training (sae.py:97)
    self.register_buffer('threshold', torch.tensor(0.0))
    # P1: contador de batches sin activación por feature - para L_aux (sae.py:30-32)
    self.register_buffer('num_batches_not_active', torch.zeros(d_sae))

# -----
# P2: Preprocessing del input (sae.py:36-48)
# -----

def preprocess_input(self, x):
    if self.input_unit_norm:
        x_mean = x.mean(dim=-1, keepdim=True)

```

```

        x = x - x_mean
        x_std = x.std(dim=-1, keepdim=True)
        x = x / (x_std + 1e-5)
        return x, x_mean, x_std
    return x, None, None

def postprocess_output(self, x_reconstruct, x_mean, x_std):
    if self.input_unit_norm and x_mean is not None:
        x_reconstruct = x_reconstruct * x_std + x_mean
    return x_reconstruct

# -----
# P1: Encoding único + BatchTopK global (sae.py:100-119)
# -----

def compute_activations(self, x_cent):
    # P1: encoding ÚNICO con W_enc completo - un solo pase para todos los anchos
    # Las features 0:w son literalmente las mismas en todos los niveles (paper ec. 3)
    pre_acts = x_cent @ self.W_enc      # [batch, d_sae]
    acts      = F.relu(pre_acts)        # ReLU antes del topk (sae.py:102)

    if self.training:
        # P1: BatchTopK global sobre d_sae*batch_size valores (paper ec. 6)
        # Preserva valores originales con scatter - NO resta threshold (sae.py:104-114)
        topk_result = torch.topk(acts.flatten(), self.k * x_cent.shape[0])
        acts_topk = (
            torch.zeros_like(acts.flatten())
            .scatter(-1, topk_result.indices, topk_result.values)
            .reshape(acts.shape)
        )
        self._update_threshold(acts_topk)
    else:
        # P1: eval usa threshold persistente, no recalcula (paper ec. 7, sae.py:117)
        acts_topk = torch.where(acts > self.threshold, acts, torch.zeros_like(acts))

    return acts, acts_topk

@torch.no_grad()
def _update_threshold(self, acts_topk, lr=0.01):
    # P1: EMA del mínimo valor positivo - calibra para eval (sae.py:224-228)
    positive_mask = acts_topk > 0
    if positive_mask.any():

```



```

        min_positive = acts_topk[positive_mask].min()
        self.threshold = (1 - lr) * self.threshold + lr * min_positive

@torch.no_grad()
def _update_inactive_features(self, acts_topk):
    # P1: necesario para L_aux - trackea qué features no se activan (sae.py:60-62)
    self.num_batches_not_active += (acts_topk.sum(0) == 0).float()
    self.num_batches_not_active[acts_topk.sum(0) > 0] = 0

# -----
# Forward
# -----

def forward(self, x):
    """
    Retorna:
        outputs      : lista de (recon_w, hidden_w) para cada ancho matryoshka
                       recon_w está en espacio normalizado (= espacio original si input_unit_norm=True)
        all_acts      : pre-topk [batch, d_sae] - para L_aux
        all_acts_topk: post-topk [batch, d_sae]
        x_proc        : x preprocesado (= x si input_unit_norm=False)
    """
    x_proc, x_mean, x_std = self.preprocess_input(x)
    x_cent = x_proc - self.b_dec
    all_acts, all_acts_topk = self.compute_activations(x_cent)

    # P1: reconstrucciones por PREFIJO compartido (paper ec. 4, sae.py:149-155)
    # all_acts_topk[:, 0:w] son las mismas features en todos los niveles anidados
    # recon_w se mantiene en espacio normalizado para que la loss compare espacios iguales
    outputs = []
    for w in self.matryoshka_widths:
        hidden_w = all_acts_topk[:, :w]
        recon_w = hidden_w @ self.W_dec[:, :w] + self.b_dec
        outputs.append((recon_w, hidden_w))

    self._update_inactive_features(all_acts_topk)
    return outputs, all_acts, all_acts_topk, x_proc

# -----
# P1: Auxiliary loss para dead features (sae.py:198-221)
# -----

```

```

def get_auxiliary_loss(self, x_proc, x_reconstruct_full, all_acts):
    residual = x_proc.float() - x_reconstruct_full.float()
    aux_reconstruct = torch.zeros_like(residual)

    dead_features = self.num_batches_not_active >= self.n_batches_to_dead

    if dead_features.sum() > 0:
        acts_topk_aux = torch.topk(
            all_acts[:, dead_features],
            min(self.top_k_aux, int(dead_features.sum()))),
            dim=-1,
        )
        acts_aux = torch.zeros_like(all_acts[:, dead_features]).scatter(
            -1, acts_topk_aux.indices, acts_topk_aux.values
        )
        aux_reconstruct = aux_reconstruct + acts_aux @ self.W_dec[dead_features]

    if aux_reconstruct.abs().sum() > 0:
        return self.aux_penalty * (aux_reconstruct.float() - residual.float()).pow(2).mean()
    return torch.tensor(0.0, device=x_proc.device)

# -----
# P2: Constraint de norma unitaria con proyección del gradiente (sae.py:51-57)
# -----

@torch.no_grad()
def make_decoder_weights_and_grad_unit_norm(self):
    # P2: proyecta gradiente al espacio tangente de la esfera unitaria + normaliza
    # @torch.no_grad() evita trazar grafo interno - no impide leer .grad (sae.py:50)
    # Llamar ANTES de optimizer.step() (training.py:31)
    W_dec_normed = self.W_dec / self.W_dec.norm(dim=-1, keepdim=True)
    W_dec_grad_proj = (
        (self.W_dec.grad * W_dec_normed).sum(-1, keepdim=True) * W_dec_normed
    )
    self.W_dec.grad -= W_dec_grad_proj
    self.W_dec.data = W_dec_normed

# Crear modelo
model = MatryoshkaBatchTopKSAE(
    d_in=cfg['d_in'],
    d_sae=cfg['d_sae'],

```

```

k=cfg['k'],
matryoshka_widths=cfg['matryoshka_widths'],
n_batches_to_dead=cfg['n_batches_to_dead'],
top_k_aux=cfg['top_k_aux'],
aux_penalty=cfg['aux_penalty'],
input_unit_norm=cfg['input_unit_norm'],
).to(device)
num_params = sum(p.numel() for p in model.parameters())

model_info = pd.DataFrame({
    'Componente': [
        'Input', 'Encoder (W_enc)', 'Activación', 'Anchos matryoshka',
        'Hidden activos (por nivel)', 'Decoder (W_dec)', 'Output', 'Total params'
    ],
    'Dimensión': [
        f'{cfg["d_in"]}',
        f'{cfg["d_in"]} -> {cfg["d_sae"]} (único pase, prefijos compartidos)',
        f'BatchTopK global (k={cfg["k"]}) - threshold EMA en eval',
        str(cfg['matryoshka_widths']),
        f' {cfg["k"]} activos (promedio del batch)',
        f'{cfg["d_sae"]} -> {cfg["d_in"]} (slices de prefijo)',
        f'{cfg["d_in"]}',
        f'{num_params:,}'
    ]
})
print(model_info.to_string(index=False))

```

Componente	Dimensión
Input	512
Encoder (W_enc)	512 -> 8192 (único pase, prefijos compartidos)
Activación	BatchTopK global (k=81) - threshold EMA en eval
Anchos matryoshka	[2048, 4096, 8192]
Hidden activos (por nivel)	81 activos (promedio del batch)
Decoder (W_dec)	8192 -> 512 (slices de prefijo)
Output	512
Total params	8,389,120

6. Función de Pérdida MatryoshkaBatchTopK

Loss = mean MSE(recon_w, x) para cada ancho w matryoshka_widths +
 · L_aux

- **MSE en cada nivel:** error de reconstrucción desde cada sub-espacio matryoshka
- **Sin L1 penalty:** la sparsity está controlada por el threshold global de BatchTopK
- **Loss total = media de MSE en todos los anchos** — mantiene la escala estable y preserva el balance entre reconstrucción y aux_loss independientemente del número de anchos (alineado con Bussmann sae.py:172)
- La loss anidada fuerza a que los sub-espacios más pequeños también sean representaciones completas del input

```
def loss_function(x_proc, outputs, all_acts, model):
    """
    Pérdida MatryoshkaBatchTopK: media de MSE en cada ancho + auxiliary loss.

    Paper ec. 5:  $L(x) = \sum_{m \in M} \|x - \hat{x}\|^2 + \lambda L_{aux}$ 
    Bussmann (sae.py): promedia dividiendo por n_grupos - mantiene la escala de
    la MSE estable independientemente del número de anchos, y preserva el balance
    entre reconstrucción y aux_loss.

    Args:
        x_proc: activaciones preprocesadas [batch, d_in] (= x si input_unit_norm=False)
        outputs: lista de (recon_w, hidden_w) por ancho matryoshka
        all_acts: activaciones pre-topk [batch, d_sae] - para L_aux
        model: referencia al modelo (para get_auxiliary_loss)

    Returns:
        total_loss, mse_per_width, aux_loss
    """
    x_reconstruct_full = outputs[-1][0] # reconstrucción del ancho máximo

    # Media de MSE en todos los anchos + término b_dec (alineado con Bussmann sae.py:163-172)
    mse_bdec = F.mse_loss(model.b_dec.expand_as(x_proc), x_proc)
    mse_per_width = [F.mse_loss(recon_w, x_proc) for recon_w, _ in outputs]
    mean_mse = (mse_bdec + sum(mse_per_width)) / (len(mse_per_width) + 1)

    # P1: auxiliary loss para dead features (paper ec. 5:  $\lambda L_{aux}$ , sae.py:198-221)
    aux_loss = model.get_auxiliary_loss(x_proc, x_reconstruct_full, all_acts)

    return mean_mse + aux_loss, mse_per_width, aux_loss

# P3: betas según config.py de los autores - beta2=0.99 vs default PyTorch 0.999
# (config.py: beta1=0.9, beta2=0.99)
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=cfg["lr"],
```

```

    betas=(cfg["beta1"], cfg["beta2"])
)

optim_info = pd.DataFrame({
    'Parámetro': ['Optimizador', 'lr', 'beta1', 'beta2',
                  'k (L0 promedio)', 'Anchos matryoshka',
                  'aux_penalty', 'n_batches_to_dead', 'top_k_aux'],
    'Valor':      ['Adam', cfg["lr"], cfg["beta1"], cfg["beta2"],
                  cfg["k"], str(cfg['matryoshka_widths']),
                  cfg['aux_penalty'], cfg['n_batches_to_dead'], cfg['top_k_aux']]
})
print(optim_info.to_string(index=False))

```

Parámetro	Valor
Optimizador	Adam
lr	0.001
beta1	0.9
beta2	0.99
k (L0 promedio)	81
Anchos matryoshka	[2048, 4096, 8192]
aux_penalty	0.03125
n_batches_to_dead	20
top_k_aux	512

7. Loop de Entrenamiento

```

PATIENCE = NAMING_META['extras']['patience']
WIDTHS    = cfg['matryoshka_widths']

save_dir = Path('./saved_model')
save_dir.mkdir(exist_ok=True)
best_model_name = get_model_name(
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='best',
    extras_id=extras_id
)
BEST_CKPT = save_dir / best_model_name

```

```

history = {
    'train_total_loss': [],
    'train_aux_loss': [],
    'train_l0_sparsity': [],
    'train_fraction_active': [],
    'val_total_loss': [],
    'val_aux_loss': [],
    'val_l0_sparsity': [],
    'val_fraction_active': [],
}
for w in WIDTHS:
    history[f'val_mse_w{w}'] = []

best_val_loss = float('inf')
best_epoch = 0
epochs_no_improve = 0

print(f'Iniciando entrenamiento: {NUM_EPOCHS} épocas máximo')
print(f'Early stopping: patience={PATIENCE}')
print(f'k = {cfg["k"]} features activas (L0 promedio k por batch)')
print(f'Anchos matryoshka: {WIDTHS}')
print(f'Best checkpoint: {BEST_CKPT}')
print('=' * 70)

for epoch in range(NUM_EPOCHS):

    # ----- TRAIN
    model.train()
    t_total = t_aux = t_l0 = t_frac = 0.0

    for batch in train_loader:
        x = batch.to(device)
        optimizer.zero_grad()

        # P1: forward devuelve (outputs, all_acts, all_acts_topk, x_proc)
        outputs, all_acts, all_acts_topk, x_proc = model(x)
        total_loss, _, aux_loss_val = loss_function(x_proc, outputs, all_acts, model)
        total_loss.backward()

        # P2: proyectar gradiente al espacio tangente de la esfera + normalizar
        # ANTES de optimizer.step() (sae.py:51-57, training.py:31)
        model.make_decoder_weights_and_grad_unit_norm()

```

```

optimizer.step()
# REMOVED: model.normalize_decoder() - reemplazado por make_decoder_weights_and_grad

with torch.no_grad():
    hidden_full = outputs[-1][1]
    t_total += total_loss.item()
    t_aux    += aux_loss_val.item()
    t_l0     += (hidden_full > 0).float().sum(dim=1).mean().item()
    t_frac   += (hidden_full > 0).float().mean().item()

n = len(train_loader)
history['train_total_loss'].append(t_total / n)
history['train_aux_loss'].append(t_aux / n)
history['train_l0_sparsity'].append(t_l0 / n)
history['train_fraction_active'].append(t_frac / n)

# ----- VALIDATION
model.eval()
v_total = v_aux = v_l0 = v_frac = 0.0
v_per_width = {w: 0.0 for w in WIDTHHS}

with torch.no_grad():
    for batch in val_loader:
        x = batch.to(device)
        outputs, all_acts, all_acts_topk, x_proc = model(x)
        total_loss, mse_per_width, aux_loss_val = loss_function(x_proc, outputs, all_acts)
        hidden_full = outputs[-1][1]

        v_total += total_loss.item()
        v_aux    += aux_loss_val.item()
        v_l0     += (hidden_full > 0).float().sum(dim=1).mean().item()
        v_frac   += (hidden_full > 0).float().mean().item()
        for w, mse_w in zip(WIDTHHS, mse_per_width):
            v_per_width[w] += mse_w.item()

nv = len(val_loader)
history['val_total_loss'].append(v_total / nv)
history['val_aux_loss'].append(v_aux / nv)
history['val_l0_sparsity'].append(v_l0 / nv)
history['val_fraction_active'].append(v_frac / nv)
for w in WIDTHHS:
    history[f'val_mse_w{w}'].append(v_per_width[w] / nv)

```

```

width_str = ' | '.join(f'w{w}={history[f"val_mse_w{w}"][-1]:.4f}' for w in WIDTHS)
print(f"Época {epoch+1:3d}/{NUM_EPOCHS} | "
      f"Train: {history['train_total_loss'][-1]:.6f} | "
      f"Val: {history['val_total_loss'][-1]:.6f} | "
      f"L_aux: {history['val_aux_loss'][-1]:.6f} | "
      f"L0: {history['val_l0_sparsity'][-1]:.1f} | "
      f"{width_str}")

# Early stopping (basado en loss total de validación)
if history['val_total_loss'][-1] < best_val_loss - 1e-4:
    best_val_loss = history['val_total_loss'][-1]
    best_epoch = epoch + 1
    epochs_no_improve = 0
    torch.save(model.state_dict(), BEST_CKPT)
    save_checkpoint(
        model,
        base_path=NAMING_META['base_path'],
        variante=NAMING_META['variante'],
        tecnica=NAMING_META['tecnica'],
        capa=NAMING_META['capa'],
        juegos=NAMING_META['juegos'],
        estado='best',
        extras=NAMING_META['extras'],
        config=cfg,
        history=history,
        optimizer=optimizer
    )
    print(f' Mejor modelo guardado (época {best_epoch} | Val Loss: {best_val_loss:.6f})

else:
    epochs_no_improve += 1
    if epochs_no_improve >= PATIENCE:
        print(f'\nEarly stopping en época {epoch+1} (mejor: época {best_epoch})')
        break

# Restaurar mejor modelo
model.load_state_dict(torch.load(BEST_CKPT, weights_only=True))
print(f'\nMejor época: {best_epoch} | Mejor Val Loss: {best_val_loss:.6f}')

```

Iniciando entrenamiento: 1000 épocas máximo
 Early stopping: patience=20

k = 81 features activas (L0 promedio k por batch)

Anchos matryoshka: [2048, 4096, 8192]

Best checkpoint: saved_model\sae_matryoshkabatchtopk_matryoshkabatchtopk_l6_25000g_ex2_best.

=====

Época	1/1000	Train: 0.486454	Val: 0.433674	L_aux: 0.000000	L0: 91.9	w2048=0.2755
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 1 Val Loss: 0.433674)						
Época	2/1000	Train: 0.430746	Val: 0.430674	L_aux: 0.006179	L0: 79.0	w2048=0.2637
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 2 Val Loss: 0.430674)						
Época	3/1000	Train: 0.417939	Val: 0.419927	L_aux: 0.006272	L0: 79.8	w2048=0.2496
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 3 Val Loss: 0.419927)						
Época	4/1000	Train: 0.410927	Val: 0.415079	L_aux: 0.006340	L0: 78.8	w2048=0.2436
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 4 Val Loss: 0.415079)						
Época	5/1000	Train: 0.406401	Val: 0.410600	L_aux: 0.005721	L0: 79.4	w2048=0.2391
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 5 Val Loss: 0.410600)						
Época	6/1000	Train: 0.401845	Val: 0.404123	L_aux: 0.001927	L0: 80.0	w2048=0.2360
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 6 Val Loss: 0.404123)						
Época	7/1000	Train: 0.399067	Val: 0.404691	L_aux: 0.004236	L0: 80.0	w2048=0.2342
Época	8/1000	Train: 0.397416	Val: 0.402405	L_aux: 0.003419	L0: 80.2	w2048=0.2325
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 8 Val Loss: 0.402405)						
Época	9/1000	Train: 0.396466	Val: 0.403936	L_aux: 0.006017	L0: 80.3	w2048=0.2314
Época	10/1000	Train: 0.395129	Val: 0.397316	L_aux: 0.000000	L0: 80.1	w2048=0.2308
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 10 Val Loss: 0.397316)						
Época	11/1000	Train: 0.394020	Val: 0.396847	L_aux: 0.000754	L0: 80.6	w2048=0.2293
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 11 Val Loss: 0.396847)						
Época	12/1000	Train: 0.393247	Val: 0.395278	L_aux: 0.000000	L0: 80.8	w2048=0.2282
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 12 Val Loss: 0.395278)						
Época	13/1000	Train: 0.392420	Val: 0.395320	L_aux: 0.000000	L0: 80.2	w2048=0.2284
Época	14/1000	Train: 0.392077	Val: 0.394706	L_aux: 0.000000	L0: 80.3	w2048=0.2275
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 14 Val Loss: 0.394706)						
Época	15/1000	Train: 0.391326	Val: 0.394378	L_aux: 0.000000	L0: 80.2	w2048=0.2271
Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma						
Mejor modelo guardado (época 15 Val Loss: 0.394378)						

Época 16/1000 | Train: 0.390905 | Val: 0.393909 | L_aux: 0.000000 | L0: 80.3 | w2048=0.2264
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 16 | Val Loss: 0.393909)
 Época 17/1000 | Train: 0.390448 | Val: 0.394138 | L_aux: 0.000655 | L0: 80.3 | w2048=0.2258
 Época 18/1000 | Train: 0.390204 | Val: 0.396301 | L_aux: 0.003181 | L0: 80.4 | w2048=0.2254
 Época 19/1000 | Train: 0.389858 | Val: 0.394084 | L_aux: 0.000899 | L0: 80.0 | w2048=0.2253
 Época 20/1000 | Train: 0.389936 | Val: 0.393230 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2253
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 20 | Val Loss: 0.393230)
 Época 21/1000 | Train: 0.389643 | Val: 0.391790 | L_aux: 0.000000 | L0: 81.0 | w2048=0.2233
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 21 | Val Loss: 0.391790)
 Época 22/1000 | Train: 0.389158 | Val: 0.393952 | L_aux: 0.001948 | L0: 80.3 | w2048=0.2235
 Época 23/1000 | Train: 0.389280 | Val: 0.393722 | L_aux: 0.001701 | L0: 80.1 | w2048=0.2235
 Época 24/1000 | Train: 0.388890 | Val: 0.394079 | L_aux: 0.002344 | L0: 80.2 | w2048=0.2231
 Época 25/1000 | Train: 0.388355 | Val: 0.390751 | L_aux: 0.000000 | L0: 81.1 | w2048=0.2217
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 25 | Val Loss: 0.390751)
 Época 26/1000 | Train: 0.388164 | Val: 0.391555 | L_aux: 0.000646 | L0: 80.8 | w2048=0.2218
 Época 27/1000 | Train: 0.388180 | Val: 0.390439 | L_aux: 0.000000 | L0: 81.0 | w2048=0.2211
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 27 | Val Loss: 0.390439)
 Época 28/1000 | Train: 0.387981 | Val: 0.392684 | L_aux: 0.002012 | L0: 80.5 | w2048=0.2214
 Época 29/1000 | Train: 0.387450 | Val: 0.390716 | L_aux: 0.000240 | L0: 80.6 | w2048=0.2210
 Época 30/1000 | Train: 0.387452 | Val: 0.390805 | L_aux: 0.000000 | L0: 80.1 | w2048=0.2214
 Época 31/1000 | Train: 0.387142 | Val: 0.391136 | L_aux: 0.000000 | L0: 79.6 | w2048=0.2218
 Época 32/1000 | Train: 0.386933 | Val: 0.391166 | L_aux: 0.000000 | L0: 79.5 | w2048=0.2218
 Época 33/1000 | Train: 0.386630 | Val: 0.390538 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2208
 Época 34/1000 | Train: 0.386862 | Val: 0.390161 | L_aux: 0.000000 | L0: 80.3 | w2048=0.2203
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 34 | Val Loss: 0.390161)
 Época 35/1000 | Train: 0.386437 | Val: 0.389525 | L_aux: 0.000159 | L0: 81.0 | w2048=0.2191
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 35 | Val Loss: 0.389525)
 Época 36/1000 | Train: 0.386498 | Val: 0.389709 | L_aux: 0.000000 | L0: 80.4 | w2048=0.2195
 Época 37/1000 | Train: 0.386288 | Val: 0.393260 | L_aux: 0.003441 | L0: 80.3 | w2048=0.2196
 Época 38/1000 | Train: 0.386914 | Val: 0.392499 | L_aux: 0.002563 | L0: 80.0 | w2048=0.2197
 Época 39/1000 | Train: 0.386053 | Val: 0.390447 | L_aux: 0.000721 | L0: 80.1 | w2048=0.2193
 Época 40/1000 | Train: 0.385910 | Val: 0.389791 | L_aux: 0.000239 | L0: 80.2 | w2048=0.2190
 Época 41/1000 | Train: 0.385796 | Val: 0.389549 | L_aux: 0.000239 | L0: 80.4 | w2048=0.2186
 Época 42/1000 | Train: 0.385737 | Val: 0.389603 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2189
 Época 43/1000 | Train: 0.385554 | Val: 0.389542 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2188
 Época 44/1000 | Train: 0.385477 | Val: 0.389071 | L_aux: 0.000000 | L0: 80.4 | w2048=0.2181

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 44 | Val Loss: 0.389071)

Época 45/1000 | Train: 0.385814 | Val: 0.389276 | L_aux: 0.000158 | L0: 80.3 | w2048=0.2181

Época 46/1000 | Train: 0.385672 | Val: 0.389202 | L_aux: 0.000320 | L0: 80.4 | w2048=0.2178

Época 47/1000 | Train: 0.385904 | Val: 0.388752 | L_aux: 0.000079 | L0: 80.6 | w2048=0.2174

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 47 | Val Loss: 0.388752)

Época 48/1000 | Train: 0.385195 | Val: 0.389017 | L_aux: 0.000080 | L0: 80.1 | w2048=0.2177

Época 49/1000 | Train: 0.385103 | Val: 0.389317 | L_aux: 0.000000 | L0: 79.7 | w2048=0.2182

Época 50/1000 | Train: 0.385056 | Val: 0.392455 | L_aux: 0.003509 | L0: 80.1 | w2048=0.2176

Época 51/1000 | Train: 0.385022 | Val: 0.388334 | L_aux: 0.000238 | L0: 80.8 | w2048=0.2165

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 51 | Val Loss: 0.388334)

Época 52/1000 | Train: 0.385159 | Val: 0.388311 | L_aux: 0.000000 | L0: 80.5 | w2048=0.2167

Época 53/1000 | Train: 0.385017 | Val: 0.391542 | L_aux: 0.003103 | L0: 80.3 | w2048=0.2168

Época 54/1000 | Train: 0.385447 | Val: 0.388443 | L_aux: 0.000160 | L0: 80.4 | w2048=0.2165

Época 55/1000 | Train: 0.384845 | Val: 0.388233 | L_aux: 0.000238 | L0: 80.7 | w2048=0.2161

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 55 | Val Loss: 0.388233)

Época 56/1000 | Train: 0.384767 | Val: 0.388805 | L_aux: 0.000079 | L0: 79.8 | w2048=0.2170

Época 57/1000 | Train: 0.384697 | Val: 0.388359 | L_aux: 0.000081 | L0: 80.3 | w2048=0.2163

Época 58/1000 | Train: 0.384673 | Val: 0.388331 | L_aux: 0.000160 | L0: 80.3 | w2048=0.2162

Época 59/1000 | Train: 0.384356 | Val: 0.387765 | L_aux: 0.000000 | L0: 80.6 | w2048=0.2156

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 59 | Val Loss: 0.387765)

Época 60/1000 | Train: 0.384385 | Val: 0.388271 | L_aux: 0.000477 | L0: 80.6 | w2048=0.2156

Época 61/1000 | Train: 0.384258 | Val: 0.388734 | L_aux: 0.000241 | L0: 79.8 | w2048=0.2164

Época 62/1000 | Train: 0.384292 | Val: 0.388011 | L_aux: 0.000158 | L0: 80.4 | w2048=0.2155

Época 63/1000 | Train: 0.384304 | Val: 0.387972 | L_aux: 0.000475 | L0: 80.7 | w2048=0.2151

Época 64/1000 | Train: 0.384048 | Val: 0.387057 | L_aux: 0.000000 | L0: 81.1 | w2048=0.2144

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 64 | Val Loss: 0.387057)

Época 65/1000 | Train: 0.384110 | Val: 0.388072 | L_aux: 0.000080 | L0: 80.0 | w2048=0.2155

Época 66/1000 | Train: 0.384069 | Val: 0.388030 | L_aux: 0.000000 | L0: 79.9 | w2048=0.2156

Época 67/1000 | Train: 0.384699 | Val: 0.387748 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2152

Época 68/1000 | Train: 0.384191 | Val: 0.389494 | L_aux: 0.001512 | L0: 79.8 | w2048=0.2154

Época 69/1000 | Train: 0.384087 | Val: 0.389361 | L_aux: 0.002056 | L0: 80.5 | w2048=0.2145

Época 70/1000 | Train: 0.383683 | Val: 0.388570 | L_aux: 0.000477 | L0: 79.6 | w2048=0.2154

Época 71/1000 | Train: 0.384094 | Val: 0.387816 | L_aux: 0.000238 | L0: 80.1 | w2048=0.2147

Época 72/1000 | Train: 0.383693 | Val: 0.388282 | L_aux: 0.000950 | L0: 80.3 | w2048=0.2144

Época 73/1000 | Train: 0.384058 | Val: 0.387573 | L_aux: 0.000634 | L0: 80.7 | w2048=0.2139

Época 74/1000 | Train: 0.383972 | Val: 0.387585 | L_aux: 0.000317 | L0: 80.3 | w2048=0.2142

Época 75/1000 | Train: 0.383557 | Val: 0.387263 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2141

Época 76/1000 | Train: 0.384039 | Val: 0.387403 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2143
 Época 77/1000 | Train: 0.383669 | Val: 0.391170 | L_aux: 0.003651 | L0: 79.9 | w2048=0.2143
 Época 78/1000 | Train: 0.383814 | Val: 0.386689 | L_aux: 0.000158 | L0: 80.9 | w2048=0.2131
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 78 | Val Loss: 0.386689)
 Época 79/1000 | Train: 0.383950 | Val: 0.389177 | L_aux: 0.001668 | L0: 80.0 | w2048=0.2143
 Época 80/1000 | Train: 0.383420 | Val: 0.387939 | L_aux: 0.000399 | L0: 79.7 | w2048=0.2142
 Época 81/1000 | Train: 0.383182 | Val: 0.386641 | L_aux: 0.000000 | L0: 80.7 | w2048=0.2131
 Época 82/1000 | Train: 0.383410 | Val: 0.389830 | L_aux: 0.003081 | L0: 80.5 | w2048=0.2132
 Época 83/1000 | Train: 0.383162 | Val: 0.386727 | L_aux: 0.000238 | L0: 80.7 | w2048=0.2128
 Época 84/1000 | Train: 0.383816 | Val: 0.387146 | L_aux: 0.001024 | L0: 81.1 | w2048=0.2123
 Época 85/1000 | Train: 0.383754 | Val: 0.386748 | L_aux: 0.000158 | L0: 80.5 | w2048=0.2128
 Época 86/1000 | Train: 0.383471 | Val: 0.386733 | L_aux: 0.000318 | L0: 80.6 | w2048=0.2125
 Época 87/1000 | Train: 0.382940 | Val: 0.386843 | L_aux: 0.001099 | L0: 81.4 | w2048=0.2117
 Época 88/1000 | Train: 0.383049 | Val: 0.387563 | L_aux: 0.000240 | L0: 79.7 | w2048=0.2137
 Época 89/1000 | Train: 0.382918 | Val: 0.387280 | L_aux: 0.000476 | L0: 80.1 | w2048=0.2130
 Época 90/1000 | Train: 0.383192 | Val: 0.386645 | L_aux: 0.000396 | L0: 80.6 | w2048=0.2122
 Época 91/1000 | Train: 0.383159 | Val: 0.387163 | L_aux: 0.000634 | L0: 80.4 | w2048=0.2125
 Época 92/1000 | Train: 0.383413 | Val: 0.388440 | L_aux: 0.001816 | L0: 80.2 | w2048=0.2126
 Época 93/1000 | Train: 0.383119 | Val: 0.387216 | L_aux: 0.000158 | L0: 79.7 | w2048=0.2131
 Época 94/1000 | Train: 0.382952 | Val: 0.386603 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2125
 Época 95/1000 | Train: 0.383033 | Val: 0.386256 | L_aux: 0.000236 | L0: 80.7 | w2048=0.2117
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 95 | Val Loss: 0.386256)
 Época 96/1000 | Train: 0.383068 | Val: 0.386513 | L_aux: 0.000237 | L0: 80.5 | w2048=0.2121
 Época 97/1000 | Train: 0.382617 | Val: 0.386485 | L_aux: 0.000238 | L0: 80.4 | w2048=0.2119
 Época 98/1000 | Train: 0.382943 | Val: 0.386307 | L_aux: 0.000000 | L0: 80.2 | w2048=0.2119
 Época 99/1000 | Train: 0.382512 | Val: 0.386000 | L_aux: 0.000000 | L0: 80.6 | w2048=0.2116
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 99 | Val Loss: 0.386000)
 Época 100/1000 | Train: 0.382493 | Val: 0.386943 | L_aux: 0.000158 | L0: 79.7 | w2048=0.2125
 Época 101/1000 | Train: 0.382661 | Val: 0.386945 | L_aux: 0.000396 | L0: 79.9 | w2048=0.2121
 Época 102/1000 | Train: 0.383098 | Val: 0.386137 | L_aux: 0.000079 | L0: 80.5 | w2048=0.2116
 Época 103/1000 | Train: 0.383190 | Val: 0.386687 | L_aux: 0.000556 | L0: 80.3 | w2048=0.2116
 Época 104/1000 | Train: 0.382524 | Val: 0.385939 | L_aux: 0.000079 | L0: 80.5 | w2048=0.2112
 Época 105/1000 | Train: 0.382401 | Val: 0.385791 | L_aux: 0.000392 | L0: 81.0 | w2048=0.2106
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 105 | Val Loss: 0.385791)
 Época 106/1000 | Train: 0.382621 | Val: 0.386039 | L_aux: 0.000471 | L0: 80.8 | w2048=0.2108
 Época 107/1000 | Train: 0.382374 | Val: 0.386537 | L_aux: 0.000238 | L0: 80.1 | w2048=0.2117
 Época 108/1000 | Train: 0.382363 | Val: 0.385609 | L_aux: 0.000000 | L0: 80.7 | w2048=0.2107
 Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
 Mejor modelo guardado (época 108 | Val Loss: 0.385609)

Época 109/1000		Train: 0.382100		Val: 0.385822		L_aux: 0.000157		L0: 80.7		w2048=0.2108
Época 110/1000		Train: 0.382570		Val: 0.386483		L_aux: 0.000708		L0: 80.5		w2048=0.2109
Época 111/1000		Train: 0.382732		Val: 0.386506		L_aux: 0.000079		L0: 79.9		w2048=0.2118
Época 112/1000		Train: 0.382707		Val: 0.386380		L_aux: 0.000238		L0: 80.0		w2048=0.2113
Época 113/1000		Train: 0.382556		Val: 0.386749		L_aux: 0.000946		L0: 80.4		w2048=0.2109
Época 114/1000		Train: 0.382364		Val: 0.387722		L_aux: 0.001190		L0: 79.6		w2048=0.2117
Época 115/1000		Train: 0.382660		Val: 0.385657		L_aux: 0.000316		L0: 80.9		w2048=0.2103
Época 116/1000		Train: 0.382306		Val: 0.387170		L_aux: 0.000792		L0: 79.8		w2048=0.2116
Época 117/1000		Train: 0.382240		Val: 0.386650		L_aux: 0.000788		L0: 80.2		w2048=0.2108
Época 118/1000		Train: 0.382649		Val: 0.385200		L_aux: 0.000471		L0: 81.5		w2048=0.2093

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 118 | Val Loss: 0.385200)

Época 119/1000		Train: 0.382214		Val: 0.386091		L_aux: 0.000394		L0: 80.4		w2048=0.2106
Época 120/1000		Train: 0.382081		Val: 0.385553		L_aux: 0.000314		L0: 80.8		w2048=0.2099
Época 121/1000		Train: 0.382857		Val: 0.385729		L_aux: 0.000079		L0: 80.4		w2048=0.2105
Época 122/1000		Train: 0.382044		Val: 0.385921		L_aux: 0.000079		L0: 80.1		w2048=0.2107
Época 123/1000		Train: 0.382192		Val: 0.386118		L_aux: 0.000394		L0: 80.2		w2048=0.2105
Época 124/1000		Train: 0.382061		Val: 0.385902		L_aux: 0.000394		L0: 80.4		w2048=0.2102
Época 125/1000		Train: 0.382192		Val: 0.385087		L_aux: 0.000157		L0: 81.1		w2048=0.2095

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

Mejor modelo guardado (época 125 | Val Loss: 0.385087)

Época 126/1000		Train: 0.381990		Val: 0.386812		L_aux: 0.001883		L0: 81.0		w2048=0.2094
Época 127/1000		Train: 0.382195		Val: 0.385774		L_aux: 0.000237		L0: 80.3		w2048=0.2102
Época 128/1000		Train: 0.382113		Val: 0.385641		L_aux: 0.000471		L0: 80.7		w2048=0.2097
Época 129/1000		Train: 0.381875		Val: 0.386677		L_aux: 0.001569		L0: 80.8		w2048=0.2096
Época 130/1000		Train: 0.382214		Val: 0.386954		L_aux: 0.001260		L0: 80.1		w2048=0.2103
Época 131/1000		Train: 0.381971		Val: 0.386502		L_aux: 0.000240		L0: 79.5		w2048=0.2110
Época 132/1000		Train: 0.382141		Val: 0.386131		L_aux: 0.001254		L0: 81.0		w2048=0.2092
Época 133/1000		Train: 0.381632		Val: 0.385525		L_aux: 0.000706		L0: 81.0		w2048=0.2091
Época 134/1000		Train: 0.381906		Val: 0.387134		L_aux: 0.000636		L0: 79.3		w2048=0.2112
Época 135/1000		Train: 0.382190		Val: 0.387076		L_aux: 0.001028		L0: 79.7		w2048=0.2106
Época 136/1000		Train: 0.381798		Val: 0.386011		L_aux: 0.001098		L0: 80.8		w2048=0.2092
Época 137/1000		Train: 0.381828		Val: 0.386206		L_aux: 0.000788		L0: 80.2		w2048=0.2098
Época 138/1000		Train: 0.381787		Val: 0.385917		L_aux: 0.000551		L0: 80.4		w2048=0.2097
Época 139/1000		Train: 0.381699		Val: 0.385301		L_aux: 0.000315		L0: 80.7		w2048=0.2092
Época 140/1000		Train: 0.381948		Val: 0.390379		L_aux: 0.005718		L0: 81.1		w2048=0.2088
Época 141/1000		Train: 0.382395		Val: 0.385518		L_aux: 0.000394		L0: 80.5		w2048=0.2093
Época 142/1000		Train: 0.382168		Val: 0.387552		L_aux: 0.001425		L0: 79.5		w2048=0.2106
Época 143/1000		Train: 0.381750		Val: 0.385273		L_aux: 0.000470		L0: 80.8		w2048=0.2089
Época 144/1000		Train: 0.382069		Val: 0.388667		L_aux: 0.003695		L0: 80.6		w2048=0.2091
Época 145/1000		Train: 0.382214		Val: 0.386592		L_aux: 0.001419		L0: 80.3		w2048=0.2093

Early stopping en época 145 (mejor: época 125)

Mejor época: 125 | Mejor Val Loss: 0.385087

8. Guardar Modelo

```
# Guardar checkpoint final
checkpoint = {
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'config': {
        'architecture': cfg['architecture'],
        'd_in': cfg['d_in'],
        'd_sae': cfg['d_sae'],
        'k': cfg['k'],
        'matryoshka_widths': cfg['matryoshka_widths'],
        'lr': cfg['lr'],
        'hook_name': cfg['hook_name'],
    },
    'history': history,
}

final_model_name = get_model_name(
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='final',
    extras_id=extras_id
)
checkpoint_path = save_dir / final_model_name
torch.save(checkpoint, checkpoint_path)
print(f'Modelo guardado: {checkpoint_path}')

save_checkpoint(
    model,
    base_path=NAMING_META['base_path'],
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='final',
```

```

    extras=NAMING_META['extras'],
    config=cfg,
    history=history,
    optimizer=optimizer
)

```

Modelo guardado: saved_model\sae_matryoshkabatchtopk_matryoshkabatchtopk_l6_25000g_ex2_final

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

WindowsPath('C:/Users/Esposa/Documents/Repos/sae-othello-gpt/layer_06/models/sae_matryoshkab

9. Métricas básicas del modelo

```

metrics_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

training_metrics = {
    # Calidad de reconstrucción (loss total = media de MSE en todos los anchos)
    'val_total_loss_final': history['val_total_loss'][-1],
    'val_total_loss_best': best_val_loss,
    # MSE por ancho matryoshka (último epoch)
    **{f'val_mse_w{w}_final': history[f'val_mse_w{w}'][-1] for w in WIDTHS},
    # Sparsity (L0 del ancho máximo)
    'val_l0_final': history['val_l0_sparsity'][-1],
    'val_fraction_active_final': history['val_fraction_active'][-1],
    # Contexto del entrenamiento
    'best_epoch': best_epoch,
    'total_epochs': len(history['train_total_loss']),
    'train_total_loss_final': history['train_total_loss'][-1],
    'train_l0_final': history['train_l0_sparsity'][-1],
    # Hiperparámetros MatryoshkaBatchTopK
    'architecture': cfg['architecture'],
    'k': cfg['k'],
    'matryoshka_widths': cfg['matryoshka_widths'],
}

```

```

    'd_sae':                cfg['d_sae'],
    'd_in':                 cfg['d_in'],
    'expansion_factor':     cfg['d_sae'] // cfg['d_in'],
    'hook_name':            cfg['hook_name'],
}

os.makedirs(metrics_dir, exist_ok=True)
metrics_path = Path(metrics_dir) / 'training_metrics.json'
with open(metrics_path, 'w') as f:
    json.dump(training_metrics, f, indent=2)

print(f'Métricas guardadas: {metrics_path}')
print(pd.DataFrame(
    list(training_metrics.items()), columns=['Métrica', 'Valor']
).to_string(index=False))

```

Métricas guardadas: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\s

	Métrica	Valor
val_total_loss_final		0.386592
val_total_loss_best		0.385087
val_mse_w2048_final		0.209287
val_mse_w4096_final		0.194009
val_mse_w8192_final		0.183835
val_l0_final		80.342462
val_fraction_active_final		0.009807
best_epoch		125
total_epochs		145
train_total_loss_final		0.382214
train_l0_final		81.0
architecture	matryoshkabatchtopk	
k		81
matryoshka_widths	[2048, 4096, 8192]	
d_sae		8192
d_in		512
expansion_factor		16
hook_name	blocks.5.hook_resid_post	

10. Visualización de Resultados


```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Total Nested Loss
axes[0].plot(history['train_total_loss'], label='Train', color='steelblue', alpha=0.8)
axes[0].plot(history['val_total_loss'], label='Validación', color='tomato', alpha=0.8)
if best_epoch > 0:
    axes[0].axvline(best_epoch - 1, color='green', linestyle='--', alpha=0.7,
                    label=f'Best (época {best_epoch})')
axes[0].set_title(f'Total Nested Loss ( $\Sigma$  MSE en {len(WIDTHS)} anchos)')
axes[0].set_xlabel('Época')
axes[0].set_ylabel('Loss total')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

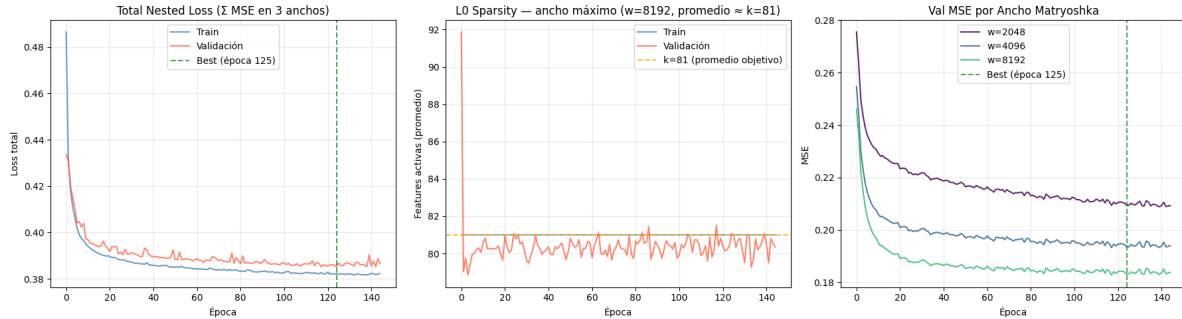
# L0 sparsity (ancho máximo, L0 promedio k)
axes[1].plot(history['train_l0_sparsity'], label='Train', color='steelblue', alpha=0.8)
axes[1].plot(history['val_l0_sparsity'], label='Validación', color='tomato', alpha=0.8)
axes[1].axhline(cfg["k"], color='orange', linestyle='--', alpha=0.8,
                label=f'k={cfg["k"]} (promedio objetivo)')
axes[1].set_title(f'L0 Sparsity - ancho máximo (w={cfg["d_sae"]}, promedio k={cfg["k"]})')
axes[1].set_xlabel('Época')
axes[1].set_ylabel('Features activas (promedio)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# MSE por ancho matryoshka (validación)
colors_w = plt.cm.viridis([i / len(WIDTHS) for i in range(len(WIDTHS))])
for w, color in zip(WIDTHS, colors_w):
    axes[2].plot(history[f'val_mse_w{w}'], label=f'w={w}', color=color, alpha=0.85)
if best_epoch > 0:
    axes[2].axvline(best_epoch - 1, color='green', linestyle='--', alpha=0.7,
                    label=f'Best (época {best_epoch})')
axes[2].set_title('Val MSE por Ancho Matryoshka')
axes[2].set_xlabel('Época')
axes[2].set_ylabel('MSE')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(plots_dir / 'training_curves.png', dpi=300, bbox_inches='tight')
plt.show()

```

```
print('\n' + '='*50)
print('Gráficas guardadas exitosamente')
print('='*50)
```



=====

Gráficas guardadas exitosamente

=====

12. Generar PDF con Quarto

Una vez ejecutado todo el notebook, ejecutar el siguiente comando en la terminal para generar el PDF con Quarto:

```
exp_dir = get_experiment_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

pdf_name = get_report_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'training',
    extras_id=extras_id
)
```

```

)

notebook_name = 'train_sae_matryoska-batch-topk.ipynb'
output_path = exp_dir / pdf_name

print(f'Generando PDF con Quarto...')
print(f'Archivo de salida: {output_path}')

# Quarto no acepta rutas en --output, solo nombre de archivo
# Se usa --output-dir para el directorio y --output solo para el nombre
result = subprocess.run(
    f'quarto render {notebook_name} --to pdf --output-dir "{exp_dir}" --output {pdf_name}',
    shell=True,
    capture_output=True,
    text=True
)

if result.returncode == 0:
    print(f'PDF generado exitosamente: {output_path}')
else:
    print(f'Error al generar PDF:')
    print(result.stderr)

```

Generando PDF con Quarto...

Archivo de salida: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae

PDF generado exitosamente: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\lay